



## week10

# LLaMA: Open and Efficient Foundation Language Models

저자: Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet 외 (Meta AI)

발행: arXiv (2023)

## 핵심 기여

- LLaMA (Large Language Model Meta AI)라는 **효율적인 대형 언어 모델(LLM) 시리즈**를 제안.
- **7B~65B(억 개) 파라미터 모델**을 개발하여, 기존 모델보다 적은 연산량으로 뛰어난 성능을 달성.
- 모든 데이터가 공개된 소스에서 수집됨 → 상업적 데이터 없이 최상위 성능을 달성.
- LLaMA-13B가 GPT-3(175B)보다 더 나은 성능을 보이며, LLaMA-65B는 Chinchilla-70B 및 PaLM-540B와 경쟁 가능.
- 연구 커뮤니티에 공개하여, 누구나 대형 언어 모델을 연구할 수 있도록 함.

## 1. 기존 문제점과 연구 배경

### (1) 기존 문제점

- 기존의 LLM(예: GPT-3, PaLM)은 과도하게 크고, 학습 비용이 높음.
- 모델 성능 향상을 위해 데이터 크기 증가보다 파라미터 크기 증가에 집중해 왔음.
- 하지만 Hoffmann et al.(2022) 연구에 따르면, 최적의 모델 크기와 데이터량 균형이 필요함.
- 기존 모델은 대부분 비공개 데이터(예: 책, 소셜미디어)를 활용하여 학습, 접근성이 제한적.

### (2) 연구 목표

- ✓ 작은 모델 크기로도 기존 거대 모델과 경쟁 가능하도록 학습.
- ✓ 공개 데이터만으로 훈련하여, 연구 커뮤니티가 자유롭게 연구 가능하도록 함.
- ✓ 적은 GPU 자원으로도 강력한 성능을 내는 모델을 제공.

## 2. LLaMA 모델 구조 및 학습 방법

### (1) 모델 크기

- **7B, 13B, 33B, 65B 파라미터 모델** 제공
- 기존 GPT-3(175B)보다 훨씬 작음에도 불구하고 높은 성능을 보임.

## (2) 학습 데이터셋

공개 데이터만 활용하여 학습!

| 데이터 출처                    | 비율   | 특징                              |
|---------------------------|------|---------------------------------|
| CommonCrawl               | 67%  | 웹 크롤링 데이터, 중복 제거 및 필터링          |
| C4                        | 15%  | 웹 문서 필터링 데이터                    |
| Github                    | 4.5% | 오픈소스 코드 (Apache, BSD, MIT 라이선스) |
| Wikipedia                 | 4.5% | 다국어 백과사전                        |
| Books (Gutenberg, Books3) | 4.5% | 공공 도메인 서적                       |
| ArXiv                     | 2.5% | 과학 논문                           |
| StackExchange             | 2%   | Q&A 포럼 데이터                      |

➡ 1.4조 개의 토큰(token) 사용하여 학습 진행

## (3) 모델 아키텍처

- **Transformer 기반 모델** (GPT-3와 유사하지만 일부 개선 적용)
- 개선된 요소
  - **Pre-normalization** (GPT-3 방식) → 학습 안정성 증가
  - **SwiGLU Activation** (PaLM 방식) → ReLU보다 성능 향상
  - **Rotary Embeddings (RoPE)** (GPTNeo 방식) → 위치 정보 처리 개선

## (4) 최적화 기법

- **AdamW Optimizer** 사용 (Loshchilov & Hutter, 2017)
- **Cosine learning rate 스케줄링** 적용
- **Gradient clipping (1.0)** 사용
- **xFormers 라이브러리**를 활용한 최적화된 **attention 연산**으로 속도 증가

# 3. 실험 및 결과 분석

## (1) 평가 데이터셋

다양한 NLP 태스크에서 평가 진행.

| 태스크                    | 벤치마크 데이터셋                                |
|------------------------|------------------------------------------|
| Common Sense Reasoning | BoolQ, PIQA, SIQA, HellaSwag, WinoGrande |
| 질문 응답 (QA)             | NaturalQuestions, TriviaQA               |
| 읽기 이해 (RC)             | RACE                                     |
| 수학적 추론                 | MATH, GSM8k                              |
| 코드 생성                  | HumanEval, MBPP                          |
| 다중 과제 이해 (MMLU)        | 57개 학문 분야 (STEM, 인문학, 사회과학 포함)           |

## (2) 주요 성능 비교

- LLaMA-13B가 GPT-3(175B)보다 뛰어난 성능을 보임.
- LLaMA-65B는 Chinchilla-70B 및 PaLM-540B와 경쟁 가능.

### Common Sense Reasoning (상식적 추론) 성능

| 모델               | BoolQ       | PIQA        | SIQA        | HellaSwag   | WinoGrande  |
|------------------|-------------|-------------|-------------|-------------|-------------|
| GPT-3 (175B)     | 60.5        | 81.0        | -           | 78.9        | 70.2        |
| Chinchilla (70B) | 83.7        | 81.8        | 51.3        | 80.8        | 74.9        |
| <b>LLaMA-65B</b> | <b>85.3</b> | <b>82.8</b> | <b>52.3</b> | <b>84.2</b> | <b>77.0</b> |

### 수학적 추론 (Mathematical Reasoning) 성능

| 모델               | GSM8K (5-shot) |
|------------------|----------------|
| GPT-3 (175B)     | 29.9           |
| Chinchilla (70B) | 35.5           |
| <b>LLaMA-65B</b> | <b>39.9</b>    |

 **결론:** LLaMA는 더 작은 크기로도 최첨단(SOTA) 성능을 달성할 수 있음!

## 4. 기술적 분석 (Ablation Study)

- 사전 학습이 없으면 성능이 급격히 하락 → 사전 학습이 필수적!
- LLaMA 모델 구조에서 Transformer 개선 요소(RoPE, SwiGLU 등) 제거 시 성능 저하 발생
- Instruction tuning을 적용하면 성능이 더욱 향상됨

## 5. 결론 및 향후 연구 방향

- 작은 모델로도 강력한 성능을 내는 효율적인 LLM 설계를 제시.
- 공개 데이터만으로 기존 대형 모델을 능가하는 성능을 달성.
- Instruction Tuning을 추가하면 성능이 더욱 향상 가능.
- 향후 더 큰 모델, 더 다양한 공개 데이터 활용 가능성 탐색 예정.

# An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

저자: Shaojie Bai, J. Zico Kolter, Vladlen Koltun

발행: arXiv (2018)

코드: [GitHub 링크](#)

## 핵심 기여

- 순차 데이터 모델링에서 RNN보다 CNN(Temporal Convolutional Networks, TCN)이 더 효과적일 수 있음을 실험적으로 검증.
- LSTM, GRU 같은 대표적인 순환 신경망(RNN)과 비교하여 TCN이 더 나은 성능을 보이며, 더 긴 기억력을 가짐.
- 다양한 벤치마크 실험을 통해 TCN이 RNN보다 더 안정적이고 병렬 연산이 가능하며, 긴 시퀀스 학습에서 강점을 보인다고 주장.
- TCN을 RNN의 대안으로 고려할 가치가 있음을 강조.

## 1. 기존 문제점과 연구 배경

### (1) 기존 문제점

- 일반적으로 순차 데이터 모델링 = RNN 기반 모델로 생각됨.
- 하지만 최근 연구에서 CNN 기반 모델도 순차 데이터에서 좋은 성능을 보일 수 있음이 확인됨.
- 대표적인 CNN 기반 순차 모델:
  - WaveNet (van den Oord et al., 2016) – 음성 합성
  - ByteNet (Kalchbrenner et al., 2016) – 기계 번역
  - Gated Convolution (Dauphin et al., 2017) – 언어 모델링
- 그렇다면 CNN(TCN)과 RNN 중 무엇이 더 좋은가? → 실험적 비교가 필요!

### (2) 연구 목표

- ✓ TCN과 RNN을 다양한 순차 모델링 태스크에서 정량적으로 비교.
- ✓ TCN이 RNN보다 더 긴 기억력(long-term memory)을 가질 수 있는지 평가.
- ✓ TCN이 범용적인 시퀀스 모델링의 대안이 될 수 있는지 검토.

## 2. TCN (Temporal Convolutional Network) 모델 구조

### (1) 기본 개념

- TCN은 1D CNN 기반의 순차 모델로, 딜레이된 (dilated) 인과적(convolutional) 구조를 사용.
- "미래 정보를 참조하지 않고 과거 데이터만을 사용" → 즉, RNN처럼 시간 순서에 맞게 예측 가능.

### (2) 주요 특징

- ✓ Causal Convolutions
  - 순방향(Feed-forward) 방식으로 동작하며, 미래 정보를 참조하지 않음.
  - RNN처럼 과거 데이터만을 이용하여 현재 시점의 출력을 계산.
- ✓ Dilated Convolutions
  - 일반적인 CNN은 짧은 시간 간격만 볼 수 있음 → dilated convolution을 사용하여 긴 시퀀스도 효과적으로 학습 가능.

- dilation factor  $d$ 를 증가시키면, receptive field(참조할 수 있는 범위)가 지수적으로 증가함.

#### ✓ Residual Connections

- 네트워크가 깊어질수록 학습이 어려워지는 문제(gradient vanishing) 해결을 위해 사용.
- ResNet처럼 입력을 그대로 유지하며 변형된 정보만 추가.

## 3. 실험 및 결과 분석

### (1) 평가 데이터셋

다양한 시퀀스 모델링 태스크에서 성능 평가.

| 태스크                       | 설명                                  |
|---------------------------|-------------------------------------|
| <b>Adding Problem</b>     | 숫자 두 개를 기억하여 더하는 문제 (장기 기억력 테스트)    |
| <b>Copy Memory Task</b>   | 긴 시퀀스에서 특정 정보를 복사하는 문제 (장기 기억력 테스트) |
| <b>Sequential MNIST</b>   | MNIST 데이터를 한 픽셀씩 입력받아 분류            |
| <b>Permuted MNIST</b>     | MNIST 픽셀 순서를 랜덤으로 섞어 학습 (더 어려운 문제)  |
| <b>JSB Chorales</b>       | 바흐(Bach) 합창곡 예측 (음악 모델링)            |
| <b>Nottingham</b>         | 포크 음악 멜로디 예측                        |
| <b>PennTreebank (PTB)</b> | 단어 및 문자 수준의 언어 모델링                  |
| <b>WikiText-103</b>       | 대규모 언어 모델링                          |
| <b>LAMBADA</b>            | 문맥 기반 단어 예측 (긴 문맥 활용 능력 평가)         |

### (2) 주요 성능 비교

TCN이 대부분의 태스크에서 RNN보다 더 우수한 성능을 보임.

#### 주요 성능 비교 표

| 태스크                                 | LSTM   | GRU    | RNN    | TCN (제안 모델)   |
|-------------------------------------|--------|--------|--------|---------------|
| <b>Sequential MNIST (정확도, %)</b>    | 87.2   | 96.2   | 21.5   | <b>99.0</b>   |
| <b>Permuted MNIST (정확도, %)</b>      | 85.7   | 87.3   | 25.3   | <b>97.2</b>   |
| <b>Adding Problem (Loss ↓)</b>      | 0.164  | 5.3e-5 | 0.177  | <b>5.8e-5</b> |
| <b>Copy Memory (Loss ↓, T=1000)</b> | 0.0204 | 0.0197 | 0.0202 | <b>3.5e-5</b> |
| <b>JSB Chorales (Loss ↓)</b>        | 8.45   | 8.43   | 8.91   | <b>8.10</b>   |
| <b>PennTreebank (Perplexity ↓)</b>  | 78.93  | 92.48  | 114.50 | <b>88.68</b>  |
| <b>LAMBADA (Perplexity ↓)</b>       | 4186   | -      | 14725  | <b>1279</b>   |

✓ 결론: TCN이 RNN보다 더 높은 정확도, 더 낮은 Perplexity(혼란도)를 보이며, 더 긴 시퀀스를 학습하는데 강점을 가짐!

## 4. 기술적 분석 (Ablation Study)

### (1) 장점

- 병렬 연산 가능 (RNN은 시퀀스 순서대로 계산해야 하지만, CNN은 병렬 연산 가능)
- 긴 시퀀스 학습에 유리 (dilated convolution 덕분에 더 긴 기억력)
- Gradient Vanishing 문제 없음 (Residual Connection 사용)
- 적은 메모리 사용 (RNN보다 더 효율적)

## (2) 단점

- 추론(Inference) 시 더 많은 메모리 필요 (과거 데이터를 계속 저장해야 함)
  - 다른 태스크로 전이(Transfer) 시, 파라미터를 조정해야 할 가능성이 높음
- 

## 5. 결론 및 향후 연구 방향

- 순차 데이터 모델링에서 CNN(TCN)이 RNN보다 더 효과적일 수 있음을 실험적으로 증명.
- TCN이 LSTM, GRU보다 더 긴 시퀀스를 학습하는 데 강점이 있음.
- 이제 RNN만이 순차 모델링의 정답이 아님 → CNN 기반 접근 방식도 고려해야 함!